



UNIVERSIDADE ESTADUAL DE CAMPINAS
INSTITUTO DE COMPUTAÇÃO

MC714 — Sistemas Distribuídos
2º Semestre de 2026

Trabalho 2

Implementação de Algoritmos Distribuídos

Carlos Alberto Astudillo Trujillo (Professor)
Diogo Maciel da Cunha (Estudante de Mestrado – PED)

Campinas, Outubro de 2026

1 Descrição Geral

Esta avaliação concentra-se na implementação e na avaliação experimental de algoritmos clássicos de sistemas distribuídos. O objetivo é desenvolver soluções corretas, explicitar as hipóteses sob as quais elas operam e investigar o seu comportamento diante de variações de carga, topologia e falhas. O trabalho deverá ser desenvolvido em **duplas**.

O ambiente de referência será uma única máquina executando contêineres, conforme detalhado na Seção 2. A nuvem privada do Instituto de Computação (IC), quando disponível, será uma alternativa opcional, sem efeito sobre a nota.

A entrega no Classroom consistirá no próprio relatório em PDF, cuja primeira página deverá conter:

1. Nome e RA dos participantes.
2. Link para o repositório do trabalho no GitLab da disciplina.
3. Link para o vídeo de explicação e demonstração da solução.

O repositório deverá conter o código-fonte, os testes, a configuração do ambiente, os scripts e dados dos experimentos, as instruções de reprodução e uma cópia do relatório, conforme a Seção 3.

Data de entrega

A data de entrega é no dia **13 de novembro de 2026**. Deverão subir os arquivos no Classroom dentro da tarefa do projeto. Não precisam duplicar as submissões, apenas uma pessoa por grupo deverá submeter o projeto. Recomenda-se submeter os trabalhos com antecedência e não esperar aos últimos minutos para a submissão.

2 Implementação

A equipe deverá implementar **dois algoritmos distribuídos complementares**. Um deles deverá pertencer aos temas de ordenação de eventos, exclusão mútua, eleição ou snapshot; o segundo deverá pertencer aos temas de consenso, compromisso atômico, replicação, detecção de falhas, sistemas P2P ou coordenação distribuída. Um terceiro algoritmo poderá ser implementado como atividade de bônus.

A lista abaixo, organizada de acordo com os temas abordados por van Steen e Tanenbaum [1], apresenta possibilidades. Outros algoritmos poderão ser escolhidos mediante acordo prévio com o professor ou com o PED.

- **Sincronização de Relógios e Ordenação de Eventos:**
  Relógio Lógico de Lamport e *Relógios Vetoriais*.
  Multicast Totalmente Ordenado (baseado em relógios de Lamport) e *Multicast Confiável*.
  Algoritmo de Cristian e *Algoritmo de Berkeley* para sincronização de relógios físicos.
  Snapshot Distribuído (algoritmo de Chandy-Lamport).
- **Exclusão Mútua Distribuída:** Algoritmo Centralizado, Algoritmo de Ricart e Agrawala, Algoritmo Descentralizado (baseado em votação) e Token Ring.
- **Eleição de Líder:** Algoritmo do Anel e Algoritmo do Valentão (Bully).
- **Consenso e Compromisso Atômico:** Protocolo de Consenso simples; *Paxos*; *Raft*; *Two-Phase Commit* (2PC) e *Three-Phase Commit* (3PC); Tolerância a Falhas Bizantinas (por exemplo, uma versão simplificada do PBFT).
- **Replicação e Consistência:** *Epidemic (Gossip) Replication* (*anti-entropy* e *rumor spreading*); Replicação com protocolos baseados em primário-*backup*; Replicação baseada em *quorum*.
- **Detecção de Falhas:** Detectores de falhas baseados em *heartbeat* ou em *gossip*.
- **Sistemas P2P e Nomeação:** *Distributed Hash Table* (Chord). Lookup de chaves em $\mathcal{O}(\log N)$.

- **Agendamento, Balanceamento e Filas:** *Leaky/Token Bucket* distribuído.

Bibliotecas de comunicação, serialização, persistência e observabilidade poderão ser utilizadas. Não será permitido contabilizar como implementação um componente ou biblioteca que já implemente o algoritmo escolhido ou a sua propriedade principal. Por exemplo, a garantia de entrega oferecida por um broker não constitui, por si só, uma implementação de multicast confiável.

Modelo do Sistema e Correção

Para cada algoritmo, a equipe deverá declarar e justificar:

- o modelo temporal adotado: síncrono, assíncrono ou parcialmente síncrono;
- as propriedades dos canais, incluindo ordenação, perdas, duplicação e garantias de entrega;
- os tipos de falha considerados, como *crash-stop*, *crash-recovery*, partições ou falhas bizantinas;
- as hipóteses relativas a relógios, persistência e reinicialização dos nós;
- as propriedades de segurança e vivacidade que caracterizam uma execução correta.

As propriedades de correção deverão ser demonstradas por testes automatizados ou por verificações reproduzíveis dos logs.

A quantidade mínima de nós deverá respeitar o algoritmo e o cenário.

Tecnologias de Comunicação

O sistema de comunicação é de livre escolha. gRPC é uma opção apropriada para RPC, mas os clientes deverão configurar *deadlines* ou *timeouts* explícitos, pois as chamadas não possuem um prazo por padrão. Filas e sistemas *publish/subscribe* poderão ser empregados, desde que a equipe descreva e trate suas semânticas de entrega:

- Redis Pub/Sub e Core NATS fornecem entrega *at-most-once*; mensagens podem ser perdidas quando o consumidor está desconectado;
- Redis Streams e NATS JetStream oferecem persistência e garantias mais fortes;
- no RabbitMQ, entrega confiável exige configuração adequada de acknowledgements dos consumidores e publisher confirms.

O uso de Redis com scripts atômicos é permitido, mas uma única instância Redis constitui coordenação centralizada e um ponto único de falha, que deverá ser explicitamente discutido.

Ambiente de Programação e Testes

O trabalho poderá ser implementado em qualquer linguagem de programação. O ambiente de referência será uma única máquina executando contêineres compatíveis com OCI e orquestrados por **Docker Compose v2**. Cada nó lógico deverá executar como um processo ou contêiner independente e comunicar-se exclusivamente pela interface de rede definida pela aplicação.

O repositório deverá conter um arquivo `compose.yaml` e comandos documentados para construir, iniciar, testar e remover todo o ambiente. Docker Desktop, Docker Engine e Podman poderão ser utilizados durante o desenvolvimento. Para fins de reprodução e correção, prevalecerá o comportamento obtido no ambiente de referência divulgado pela disciplina.

As alternativas de execução são:

- **Execução local (referência):** todos os nós são executados em uma única máquina por meio do arquivo `compose.yaml`.

- **Nuvem privada do IC (opcional):** quando disponibilizada, poderá ser usada sem alterar os requisitos ou a avaliação. A mesma implementação deverá funcionar nos dois ambientes mediante alterações exclusivamente de configuração.

O sistema deverá possuir no mínimo **três nós lógicos**, ou quantidade maior quando exigida pelo algoritmo ou pelo cenário de falha estudado. Contêineres executados em um único computador representam nós lógicos, mas não máquinas ou domínios de falha independentes; essa limitação deverá ser considerada na análise.

Falhas e condições de rede poderão ser produzidas no nível da aplicação ou por ferramentas como Toxiproxy, para conexões TCP, e `tc/netem`, em ambientes Linux que permitam a sua execução. A escolha deverá ser documentada e os cenários deverão ser repetíveis, preferencialmente por meio de scripts e sementes pseudoaleatórias registradas.

Análise Experimental (Obrigatória)

A correção e o desempenho serão avaliados separadamente. Para **cada algoritmo implementado**, a equipe deverá:

- demonstrar as propriedades de correção em pelo menos um cenário normal e um cenário de falha;
- definir e justificar pelo menos uma métrica de desempenho relevante, como latência, vazão, quantidade de mensagens, tempo de convergência, número de saltos ou tempo de recuperação;
- variar pelo menos um parâmetro relevante em três ou mais níveis, como número de nós, carga, atraso ou perda de mensagens;
- repetir cada configuração com execuções ou sementes independentes e apresentar medidas de tendência e variabilidade; para latências, deverão ser incluídos percentis, como p50, p95 ou p99;
- registrar duração dos ensaios, aquecimento, hardware, sistema operacional, versões das ferramentas e configuração utilizada;
- analisar os resultados à luz do comportamento esperado e da complexidade de mensagens ou computacional do algoritmo.

Os experimentos deverão ser automatizados e reproduzíveis. Resultados absolutos obtidos em computadores diferentes não serão comparados diretamente na avaliação.

3 Entregáveis

1. **Relatório:** documento em formato IEEE, duas colunas e no máximo seis páginas, excluindo as referências. Deverá descrever motivação, modelo do sistema, arquitetura, algoritmos, propriedades de correção, testes, cenários experimentais, métricas, resultados, limitações e conclusões. É obrigatório citar as fontes de qualquer código reutilizado e descrever as modificações realizadas.
2. **Repositório:** código-fonte, testes, `compose.yaml`, instruções de reprodução, scripts dos experimentos, dados brutos e uma cópia do relatório. A versão avaliada será identificada por uma tag de entrega definida pela disciplina.
3. **Vídeo:** gravação de aproximadamente cinco minutos, com a participação dos dois integrantes, demonstrando pelo menos um cenário normal, uma falha injetada e a recuperação ou consequência esperada.

4 Avaliação

A avaliação do projeto será baseada nos critérios e pesos detalhados na tabela abaixo.

Tabela 1: Critérios de Avaliação do Projeto

Critério	**Ponderação (%)**
Correção dos algoritmos e testes	30
Modelo, arquitetura e tratamento de falhas	20
Experimentos e reprodutibilidade	25
Relatório	15
Vídeo	10
Total	**100**

Itens Extras (Bônus)

Implementações que explorem conceitos adicionais poderão receber até **10 pontos adicionais na pontuação bruta**, respeitado o teto de nota definido pela disciplina. São exemplos:

- um terceiro algoritmo distribuído, além dos dois exigidos;
- cenários adicionais de recuperação, partição ou falhas combinadas;
- integração dos algoritmos para resolver um problema prático coerente;
- execução comparativa no ambiente local e na nuvem privada, quando esta estiver disponível.

Referências

[1] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed., distributed-systems.net, 2023.